

Prediction Terminal — Guía para el equipo

Documento para que todo el equipo entienda **qué construimos, qué pedía el curso y cómo lo cumplimos**, listo para la presentación final.

1. ¿Qué es el proyecto? (el pitch en 30 segundos)

Prediction Terminal es un servicio web tipo "terminal financiera" que conecta **mercados de predicción** (Polymarket, Kalshi) con **modelos cuantitativos de retornos de acciones**.

La idea central: los precios de los mercados de predicción son probabilidades (ej. "¿la Fed baja tasas en junio?"). Nosotros las tratamos como **factores** y medimos cómo explican el movimiento de acciones, además de buscar oportunidades de trading entre venues.

Tiene **tres modos** en una sola app:

- 1. **Regression** — ajusta modelos de factores (OLS robusto) de retornos de acciones sobre factores derivados de mercados de predicción.
- 2. **Strategies (a Hub)** — estrategias de alpha curadas y validadas: cointegración, pares, bandas OU, arbitraje cross-venue, crypto micro.
- 3. **Terminal** — un data hub estilo Bloomberg: noticias, comparables, simulación de portafolio, superficies de volatilidad, densidades de probabilidad implícitas.

2. Los requisitos del curso y CÓMO los cumplimos

El curso (ver [research/PLAN.md](#) §2) pedía 10 cosas. Aquí está cada una, cómo la cumplimos y **dónde verlo**:

#	Requisito	¿Cómo lo cumplimos?	Dónde verlo
1	Repo en GitHub con commits	Repo privado DGallardoL/predictionterminal , todo el código versionado	github.com/DGallardoL/predictionterminal
2	CI/CD automatizado (lint + test + build)	GitHub Actions con 4 jobs: lint (ruff), type-check (mypy), tests (pytest + cobertura ≥70%), Docker build	.github/workflows/ci.yml
3	Documentación con la matemática ("quants")	Documento con toda la mate del modelo en LaTeX	docs/quants.md
4	ADRs (decisiones de arquitectura)	18 ADRs (pedían 6-7), uno por decisión técnica real	docs/adrs/
5	OpenAPI auto-generado	FastAPI lo genera solo; 321 endpoints documentados	http://localhost:8000/docs y docs/openapi.json
6	Infra documentada (Docker-compose)	docker-compose para dev y producción	docker-compose.yml , docker-compose.prod.yml
7	README claro	Instrucciones, badges, ejemplos de curl, links a docs	README.md
8	Corre con <code>docker-compose up</code>	Levanta API + frontend + Redis; healthchecks incluidos	docker-compose.yml

#	Requisito	¿Cómo lo cumplimos?	Dónde verlo
9	Demo de 15 min	Guion de demostración paso a paso	<code>docs/DEMO_SCRIPT.md</code>
10	Calidad profesional ("mejor que 3")	~334 módulos Python, ~6000 tests, 18 ADRs, docs extensos, deploy en la nube	todo el repo

Resumen para presentar: los 10 requisitos están cubiertos. Los puntos fuertes para destacar son la **disciplina de ingeniería** (CI verde, tests, ADRs) y que **corre de verdad** (no es solo código, se despliega y funciona).

3. El core cuantitativo (en lenguaje simple)

Esto es lo que el profe valora como "quant honesto". Lo que hay que poder explicar:

- **Transformación logit de precios.** Un precio de mercado de predicción vive en [0,1]. Lo convertimos a logit para que se comporte como una variable de regresión sin límites. (ADR-0002)
- **Retornos logarítmicos.** `r_t = log(P_t / P_{t-1})` — estándar en finanzas, no retornos simples.
- **OLS con errores estándar robustos (HAC).** Usamos `statsmodels` con errores tipo HAC (corrigen heterocedasticidad y autocorrelación), no inventamos nuestra propia regresión. (ADR-0003)
- **VIF (factor de inflación de varianza).** Reportamos colinealidad entre factores para ser honestos sobre la calidad del ajuste.
- **Clipping configurable.** Los precios extremos (0.005, 0.002) se recortan con un epsilon explícito que el usuario puede ajustar (parámetro en `/fit`).
- **Validación de robustez 4 trimestres.** Una estrategia NO se marca "deployable" hasta pasar estabilidad de Sharpe en 4 trimestres disjuntos. Si cambia de signo o colapsa, va a la lista de "anti-alphas". (ADR-0013)

Mensaje honesto a transmitir: es un POC, no investigación de hedge fund. Pero lo que está, está bien hecho y es verificable.

4. Feature destacada: descubrimiento de arbitraje cross-venue

Esto es lo más nuevo y vistoso para la demo. Es un **data hub** (NO ejecuta trades — solo lista posibles oportunidades para que un humano las revise).

Qué hace: explora continuamente los mercados de Kalshi y Polymarket, encuentra pares que hablan del mismo evento ("¿gana X la nominación 2028?") y los muestra con su nivel de confianza.

Cómo funciona (para explicarlo):

- Un **crawler resumible** recorre miles de mercados paso a paso (guarda un checkpoint, no se limita a un número fijo).
- Modo **"New events"**: cuando aparece un mercado nuevo en un venue, lo compara contra el universo del otro venue (no solo contra lo nuevo del otro). Cada candidato se marca con un tag **NEW**.
- Modo **"Liquid"**: escanea el universo de mayor volumen de ambos venues.
- Un **matcher con filtros estrictos** (jurisdicción, umbral, ventana de resolución) marca cada par como **verified** o **review** — pero **no esconde nada** (filosofía recall-first: mejor ver un falso positivo que perderse uno real).
- Un **chequeo de precios consciente de fees** verifica si un par es realmente ejecutable.

Hallazgo honesto e importante (decirlo en la demo): el arbitraje cross-venue ejecutable de verdad es **raro**. Muchas "oportunidades" que se ven enormes (40-80%) son en realidad **mapeos mal apareados** (ej. Kalshi dice "¿queda 2º?" y Polymarket dice "¿gana?" — no son la misma apuesta). Nuestro sistema lo detecta y lo marca. Esto demuestra rigor: no vendemos humo.

Dónde verlo: Strategies → Cross-venue Arb → pestaña **Discovery**.

5. Arquitectura y stack

- **Backend:** FastAPI (Python 3.12), `statsmodels` para la regresión, Redis opcional para caché.
- **Frontend:** HTML plano + Plotly desde CDN (sin framework pesado — decisión deliberada, ADR-0009). Una sola página, tres modos.
- **Motores en segundo plano:**
 - `arbstuff/` — el motor de arbitraje (escanea Kalshi+Polymarket, escribe estado que la UI lee).
 - `cryptostuff/` — motor de microestructura crypto (opcional).
- **Infra:** Docker + docker-compose; CI en GitHub Actions; deploy en Fly.io.
- **Escala:** ~334 módulos Python, **321 endpoints**, **1,260 factores**, ~6,000 tests, 18 ADRs.

6. Cómo correrlo (para que cada quien lo pruebe)

Opción A — Docker (lo que pide el curso):

```
docker-compose up
```

Luego abrir `http://localhost:8080` (frontend) y `http://localhost:8000/docs` (API).

Opción B — local (para desarrollar):

```
cd api
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
PYTHONPATH=src uvicorn pfm.main:app --reload
```

Verificar que funciona:

```
curl http://localhost:8000/health      # {"status":"ok"}
curl http://localhost:8000/factors    # lista de factores
```

7. Guion para la presentación (15 min)

Sugerencia de reparto y orden (ver `docs/DEMO_SCRIPT.md` para el detalle):

1. **(2 min) Intro + el problema.** Qué son los mercados de predicción y por qué tratarlos como factores.
2. **(3 min) Modo Regression.** Correr un `/fit` en vivo: elegir una acción + factores, mostrar betas, t-stats, R^2 , VIF. Explicar HAC y logit.
3. **(3 min) Modo Strategies / a Hub.** Mostrar las estrategias validadas y la honestidad del tiering (validated vs review vs anti-alpha).
4. **(3 min) Cross-venue Arb Discovery.** La feature estrella: mostrar el data hub encontrando pares, explicar recall-first y el hallazgo honesto de los falsos positivos.
5. **(2 min) Ingeniería.** Mostrar CI verde, los 18 ADRs, OpenAPI auto-generado, que corre en Docker.
6. **(2 min) Cierre.** Está desplegado en la nube y accesible. Es un POC honesto y profesional.

Tip: lo que más impresiona al profe no es el quant, es la **disciplina de ingeniería**. Recalcar: CI verde, tests, Docker, ADRs genuinos, OpenAPI.

8. Deploy (en la nube)

- Configurado para **Fly.io** como una sola app que sirve API (`/`) + UI (`/ui`).
 - Config en `fly.toml` ; secretos (Redis, tokens) van por `flyctl secrets set` — **nunca** en el repo.
 - Comando: `flyctl deploy` . Dominio gratis: `pfm-prod.fly.dev` ; dominio propio con `flyctl certs add` .
-

9. Estado del repo

- **Repo:** `github.com/DGallardoL/predictionterminal` (privado).
 - **Para que el profe lo califique:** agregarlo como colaborador (Settings → Collaborators), ya que el repo es privado.
 - **Seguridad:** revisado — no hay secretos, API keys ni credenciales en el repo. Solo `.env.example` con plantillas vacías. Los archivos de estado/runtime y `.env` reales están en `.gitignore` .
-

10. Glosario rápido (por si preguntan)

- **Factor:** una probabilidad de mercado de predicción usada como variable explicativa.
 - **HAC:** errores estándar robustos a heterocedasticidad y autocorrelación.
 - **VIF:** mide si dos factores son redundantes (colinealidad).
 - **Logit:** transformación de un precio en $[0,1]$ a un número sin límites.
 - **Cross-venue arb:** comprar barato en un venue y vender caro en otro el mismo evento.
 - **Recall-first:** preferimos mostrar de más (con etiqueta) que esconder algo real.
 - **OU bands / cointegración:** técnicas de pares de trading basadas en reversión a la media.
 - **Tiering honesto:** clasificar estrategias como validadas / tentativas / anti-alpha según pruebas de robustez.
-

Cualquier duda sobre una parte específica, revisar el `README.md` , los ADRs en `docs/adrs/` , o `docs/USER_GUIDE.md` para el recorrido completo de las features.